

Contention-Aware GPU Thread Block Scheduler for Efficient GPU-SSD

Xueyang Liu*, Seonjin Na*, Euijun Chung*, Jiashen Cao*, Jing Yang†, Hyesoon Kim*

*Georgia Institute of Technology, USA

†Samsung Memory Solution Lab, USA

Emails: {xliu791, seonjin.na, echung67, jiasenc, hyesoon}@gatech.edu, jing2.yang@samsung.com

Abstract—The growing dataset sizes in LLM have made low-cost SSDs a popular solution for extending GPU memory in mobile devices. In this paper, we introduce **CA-Scheduler**, a contention-aware scheduling scheme for GPU-initiated SSD access. The key insight behind **CA-Scheduler** is leveraging the BSP GPU programming model, which allows reordering work at the thread block level to optimize SSD throughput. By capitalizing on the predictable memory access patterns of GPU thread blocks, **CA-Scheduler** anticipates SSD locations to minimize contention and improve performance.

Index Terms—GPU-SSD Architecture, Thread Block Scheduler, AI Workloads, Memory Optimization.

I. INTRODUCTION

In the current paradigm of executing AI models on mobile devices, the entire model is loaded into DRAM or HBM, a practice that significantly limits the maximum model size that can be efficiently processed on the device. For instance, a model with 10 billion parameters requires over 5GB of memory to store its parameters in INT4 format. To address the limitations of on-device memory size, extending memory with SSDs has recently been introduced. For example, “LLM in a Flash” [1] explores the use of flash memory for the efficient storage and retrieval of large language model weights. Similarly, ZnG [9] and BaM [6] investigate hardware techniques for integrating GPUs and SSDs on the same chip.

In this work, we make a key observation based on the fundamental GPU programming model: Bulk Synchronous Programming (BSP). BSP allows work to proceed concurrently until synchronization is required at the kernel level. This enables the concurrent execution of thread blocks (TBs)¹ within the same kernel in arbitrary order. While the execution order of thread blocks does not affect program correctness, it can lead to performance variations. This suggests that the scheduling order can be adjusted to optimize memory access patterns. Although GPU TB scheduling has been studied in the context of improving GPU cache locality [3], [5], there has been no prior research on leveraging TB scheduling to reduce memory system congestion. Unlike DRAM bank conflicts, which typically incur latencies on the order of 10–40 ns, congestion in SSD channels (msec order) or chips can significantly impact performance.

Based on this observation, we introduce CA-scheduler, a contention-aware GPU thread block scheduler designed to

alleviate SSD channel and chip contention. Using GPU thread block address prediction, CA-scheduler sorts the thread blocks into chip buckets and distributes requests across all SSD channels and chips.

In the context of machine learning, we observe strong stride patterns across all the workloads we evaluated, including CNN, BERT, Gemma, GPT-2, and ResNet. The memory access locations are predictable and generally localized to one or two SSD chips. These properties maximize the effectiveness of CA-scheduler, which mitigates internal SSD contention and improves SSD utilization, resulting in significant performance gains. Our evaluation shows that CA-scheduler achieves up to 2.69x speedup, with an average improvement of 1.68x compared to conventional thread block scheduling policies.

II. BACKGROUND ON SSD

Modern SSDs are harnessed with dedicated SOC with embedded firmware processor, volatile DRAM memory, and NAND flash non-volatile storage. An overview of SSD organization is shown in Figure 1. The address translation unit in SSD firmware is named as *Flash Translation Layer (FTL)*. The FTL translates *Logical Page Address (LPA)* received from the host interface to *Physical Page Address (PPA)*, which implies the location of the data as shown in Algorithm 1.

The NAND flash storage back-end is managed hierarchically. Each channel consists of one channel controller and multiple chips that share the same controller and I/O bus to transfer data. Chips are abstracted into a ‘Logical Unit’ as the basic unit for parallel access. Different channels can be accessed in parallel, while different chips in the same channel can be accessed in an interleaved manner, and the same chip cannot handle requests in parallel. This results in different kinds of resource contention: channel-level contention results in slowdown, while chip-level contention results in stalls.

Algorithm 1 CWDP page allocation scheme [8]

```

1: function PALLOC( $LPA$ )
2:    $CID \leftarrow LPA \% N_{Channel}$ 
3:    $WID \leftarrow (LPA / N_{Channel}) \% N_{Chip}$ 
4:    $DID \leftarrow (LPA / (N_{Channel} \times N_{Chip})) \% N_{Die}$ 
5:    $PID \leftarrow (LPA / (N_{Channel} \times N_{Chip} \times N_{Die})) \% N_{Plane}$ 
6: end function

```

¹Also referred to as CTAs in much of the literature

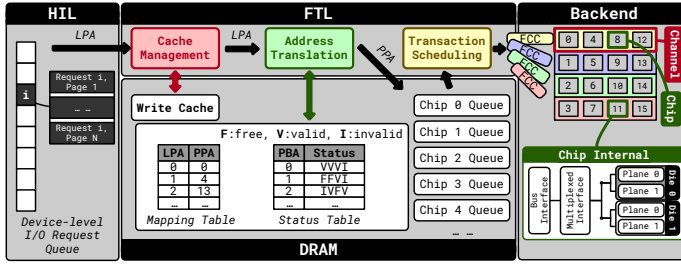


Fig. 1. SSD SoC organization, NAND storage arrays hierarchies, and interface abstraction to the host I/O stack.

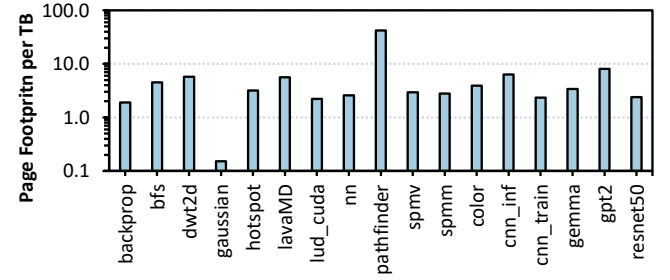


Fig. 3. Average SSD page footprint per thread block.

III. MOTIVATION

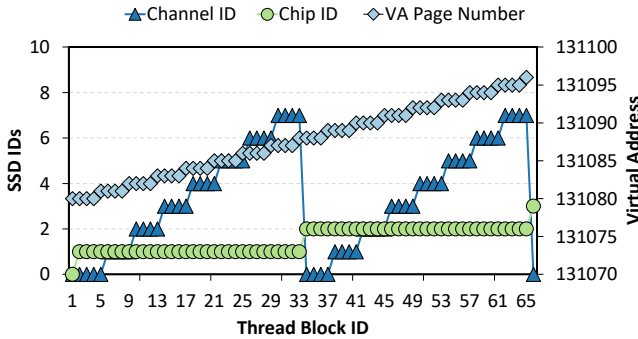


Fig. 2. Cache miss address footprint of an example benchmark and corresponding SSD channel and chip IDs, ordered by GPU thread block ID.

Thread block addresses are known to exhibit a regular stride access pattern that can be statically detected. Figure 2 provides an example from the backpropagation algorithm to illustrate this predictability. In the figure, 'VA' represents the GPU virtual page addresses accessed by each thread block, while 'channel ID' and 'chip ID' denote the physical SSD locations accessed. The SSD channel/chip IDs are determined using the CWDP page allocation scheme, as described in Algorithm 1. The results indicate that thread blocks access memory addresses sequentially, creating a linear and predictable access pattern to the SSD. To evaluate the feasibility of controlling memory access patterns through scheduling, we characterized the thread block memory footprint. Figure 3 shows the average number of SSD physical pages accessed per thread block. For most benchmarks, except for the *pathfinder* benchmark, the number of unique SSD physical pages accessed is very low.

This observation can be attributed to the GPU programming model's inherent spatial locality, where data is divided into small slices for concurrent consumption by each thread block.

If each thread block needed to access data across all SSD chips, scheduling alone would have little impact on SSD load balancing and contention. However, given that thread block has a strong locality, it is possible to categorize thread blocks based on the primary SSD locations they access. This categorization enables request load balancing across SSD chips, significantly reducing contention and improving overall system performance.

IV. CA-SCHEDULER DESIGN

A. Scheduling Effect on SSD Contention

Figure 4 illustrates how CA-scheduler operates. In this example, the GPU is assumed to have 4 streaming multiprocessors (SMs), each executing a single TB. The SSD contains 4 channels, each with 4 chips. Under a round-robin scheduling policy, TBs are picked sequentially. Since all TBs access SSD page 0, they are mapped to the same chip in the SSD, resulting in both channel and chip contention.

In Case (b), where CA-scheduler is applied and stride prediction is accurate, the GPU SMs fully utilize all SSD channels. For instance, the scheduler assigns TB4 to SM1 instead of TB1. TB4 accesses page 1, which is mapped to channel 1, and a similar redistribution occurs for the other SMs. In this ideal scenario, each SM executes a thread block mapped to a different channel, maximizing SSD throughput and resource utilization.

Case (c) demonstrates the effect of suboptimal stride selection. While the scheduler mitigates chip-level contention, incorrect stride prediction results in all pages being mapped to the same channel. Although requests to the same channel do not stall, they share a common bus for access. Consequently, latency in Case (c) improves compared to Case (a) but remains suboptimal compared to Case (b), where contention is fully eliminated.

B. CA-scheduler Algorithm

The algorithm relies on the virtual page number of TB, the stride provided by the compiler, and the computation of logical-to-physical page addresses. TBs are sorted into bins based on the SSD chips they access. To schedule new TBs, the algorithm selects one TB from each bin in a round-robin manner. A pointer tracks the next bin to pick from and increments after scheduling a thread block. Empty bins are skipped until all bins are empty, meaning no TBs remain to be scheduled.

This round-robin approach maximizes load balance and minimizes contention across SSD chips. When multiple GPU kernels are executed sequentially, the scheduling algorithm is reapplied at each kernel launch.

The GPU TB scheduler computes the channel and chip IDs based on virtual addresses, as detailed in Algorithm 1, without needing to access an address mapping table. The stride information is identified by the compiler, following an

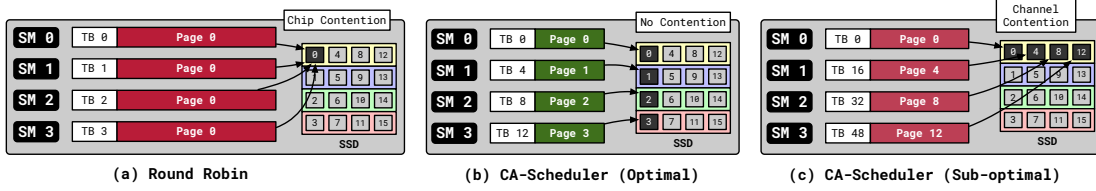


Fig. 4. Effect of scheduling policies and stride prediction on SSD contention. The GPU has 4 SMs, and the SSD has 4 channels, each with 4 chips.

Algorithm 2 Simplified Scheduling Algorithm

Data: Virtual page number (VPN_{TB_0}) initialized to 0
Result: Scheduled thread blocks
Input : Thread blocks (TB_i)
Output: Scheduled thread blocks

```

for each thread block ( $TB_i$ ) do
    Compute  $VPN_{TB_i}$  using stride. Map  $TB_i$  to SSD re-
    sources via the Palloc function. Place  $TB_i$  into the
    appropriate bin.
endfor
while unscheduled thread blocks exist do
    Select the next thread block from bins. if a processing
    unit is available then
        Schedule the selected thread block. Mark the thread
        block as scheduled;
    endif
    if  $TB_i$  completes execution then
        Update  $VPN_{TB_i}$  based on the most frequent ac-
        cess page. Reinitialize bins. Reschedule all waiting
        thread blocks.
    endif
endwhile

```

approach similar to [3]. Upon kernel launch, the GPU driver extracts the stride metadata from the binary and forwards it to the thread block scheduler. The scheduler can be implemented in various ways—for example, as a command processor in AMD-style GPUs, a dedicated hardware unit in NVIDIA-style architectures, or as part of the driver in RISC-V GPUs. The detailed implementation of the thread block scheduler is beyond the scope of this paper

C. GPU Memory System

To evaluate GPU-initiated SSD access, GPU L2 cache misses are sent directly to the SSD controller to retrieve or write data. Dirty evictions from the L2 cache are written to the SSD. A DRAM buffer within the SSD controller aggregates write requests. The GPU L2 cache line size is set to 512B, matching the SSD sector alignment size and avoiding read/write amplification caused by smaller request sizes.

V. METHODOLOGY

We use a GPU-SSD co-simulation platform, collecting GPU instruction traces with NVBit and simulating them with MacSim [4] and MQSim [7]. GPU L2 cache misses generate memory requests for MQSim, integrating GPU and SSD behavior.

TABLE I
CONFIGURATION OF SIMULATED GPU-SSD SYSTEM.

GPU Configuration [2]				
Shader Core	16 SMs, 1.0 GHz, Max 4 blocks per core			
L1 Cache	16KB, 4-way, LRU			
Shared L2 Cache	4MB, 8-way, LRU, 512B line size			
threadblock Scheduling	Round-Robin			
SSD Configuration [7]				
Host Interface	PCIe 3.0 (NVMe)			
Queue Fetch Size	512			
Channel	8 channels			
Chip	4 chips per channel			
Palloc Scheme	CWDP			
Workload	# of Blocks	# Memory Reqs	Total Reqs / Total Blocks	Major Memory Stride
backprop	16384	27650	3.38	3.76
bfs	2560	25041	9.8	1
dwt2d	342	3348	9.8	4
hotspot	1849	6144	3.3	3.64
lavaMD	125	1920	15.3	6.58
lud_cuda	1376	6576	4.8	12
nn	4096	24576	6.0	2
pathfinder	1605	211626	47.4	1
spmm	346	2915	8.4	2
spmv	253	2275	9.0	2
color	276	4053	14.7	2
cnn_inf	1440	6361	4.42	4
cnn_train	1440	2881	2.00	4
bert	371	2976	8.00	2
gemma	568	1858	3.27	8
gpt2	1393	14794	10.62	4
resnet50	1024	3168	3.09	4

Table I outlines the simulation setup. CWDP in Algorithm 1 is used for page allocation to maximize throughput, modeling the latest ARM GPUs [2]. Table V lists all the evaluated benchmarks and the total number of TBs for each benchmark.

VI. EVALUATION

A. Performance Improvement

Figure 5 compares the execution time of different thread block scheduling schemes: the conventional round-robin policy and the proposed CA-scheduler scheme.

On average, our proposed CA-scheduler achieves a $1.68\times$ speedup over the round-robin policy. This significant reduction in execution time is attributed to CA-scheduler's ability to improve SSD utilization by accounting for the SSD's internal architecture, including chips and channels, which are not considered in the round-robin approach.

CA-scheduler demonstrates consistent performance improvements across all benchmarks, with speedups reaching

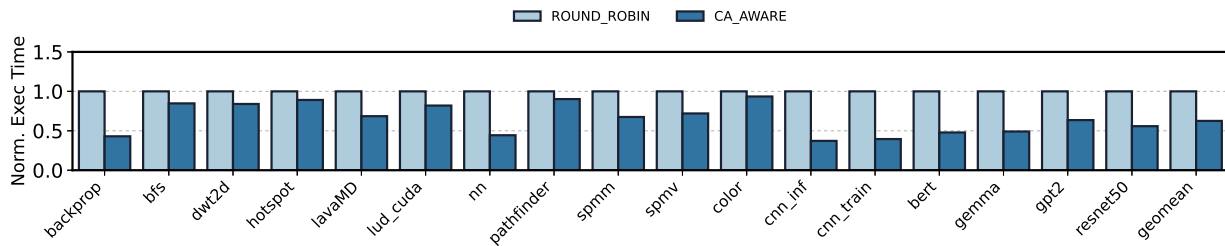


Fig. 5. Performance comparison of thread block scheduling policies. Lower values indicate better performance.

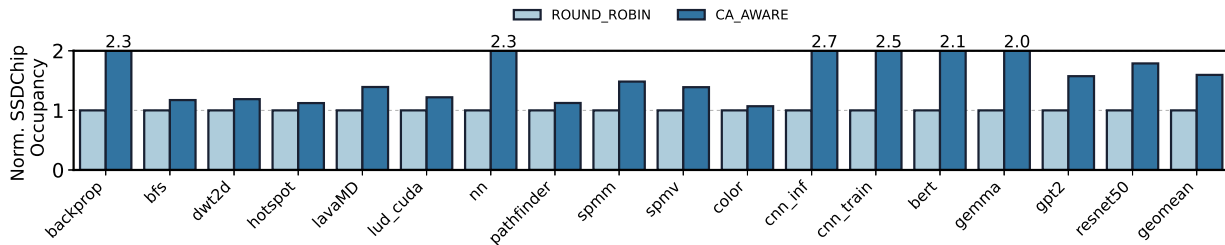


Fig. 6. SSD chip occupancy comparison across scheduling policies. Higher values indicate better utilization.

up to $2.88\times$. As shown in Figure 6, the performance improvement is directly proportional to the increase in SSD chip occupancy. Benchmarks with stronger stride patterns, such as *backprop*, *nn*, *spmm*, *spmv*, and all machine learning workloads, exhibit the greatest improvements. Specifically, ML workloads demonstrate significant speedups ranging from $1.57\times$ to $2.69\times$, owing to their strong stride patterns.

Figure 6 further highlights that reducing contention with the CA-scheduler policy results in higher SSD chip activity. This improved utilization leads to increased SSD throughput, thereby enhancing application-level, end-to-end performance.

VII. RELATED WORK

A. GPU Thread Block Scheduling

GPU thread block scheduling policies have been extensively studied, primarily to improve DRAM data locality in multi-GPU systems. CODA [5] employs a compiler-assisted approach to estimate the data utilized by each thread block and analyze access patterns for each data block. This allows for placing data alongside the thread blocks that access it. Similarly, LADM [3] leverages compiler-based analysis to co-locate thread blocks and data blocks, improving data reuse in the L2 cache.

Our scheduling policies extend similar compiler-based techniques to the context of SSD utilization. Unlike DRAM, SSD read and write latencies are significantly higher, making efficient scheduling critical for performance. While CODA and LADM target multi-GPU systems, CA-scheduler focuses on single GPU scenarios. It is possible to combine these techniques in a hierarchical scheduler to address both contexts.

VIII. CONCLUSIONS

This paper makes key contributions to GPU-SSD integration by introducing CA-scheduler, a GPU thread block scheduler

designed to mitigate SSD contention at both chip and channel levels. By leveraging the predictable memory address patterns of GPU thread blocks, CA-scheduler optimizes scheduling to enhance SSD utilization without requiring significant modifications to the existing scheduler or microarchitecture.

Our evaluation, conducted across Rodinia, Gunrock, and machine learning benchmarks, demonstrates that CA-scheduler achieves up to a $2.69\times$ speedup, with an average improvement of $1.68\times$, compared to the conventional round-robin policy. These results highlight CA-scheduler's potential to improve GPU-SSD performance in various workloads.

REFERENCES

- [1] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "Llm in a flash: Efficient large language model inference with limited memory," *arXiv preprint arXiv:2312.11514*, 2023.
- [2] armDeveloper, "Immortalis-g720: Specification," 2023, <https://developer.arm.com/Processors/Immortalis-G720> [Accessed: 04/15/2024].
- [3] M. Khairy, V. Nikiforov, D. Nellans, and T. G. Rogers, "Locality-centric data and threadblock management for massive gpus," in *2020 53rd MICRO*, 2020, pp. 1022–1036.
- [4] H. Kim, J. Lee, N. B. Lakshminarayana, J. Sim, J. Lim, and T. Pho, "Macsim: A cpu-gpu heterogeneous simulation framework user guide," *Georgia Institute of Technology*, 2012.
- [5] H. Kim, R. Hadidi, L. Nai, H. Kim, N. Jayasena, Y. Eckert, O. Kayiran, and G. Loh, "Coda: Enabling co-location of computation and data for multiple gpu systems," *TACO*, vol. 15, no. 3, pp. 1–23, 2018.
- [6] Z. Qureshi, V. S. Malthody, I. Gelado, S. W. Min, A. Masood, J. Park, J. Xiong, C. Newburn, D. Vainbrand, I. Chung *et al.*, "Bam: A case for enabling fine-grain high throughput gpu-orchestrated access to storage," *arXiv preprint arXiv:2203.04910*, 2022.
- [7] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "{MQSim}: A framework for enabling realistic studies of modern {Multi-Queue}-{SSD} devices," in *16th FAST*, 2018, pp. 49–66.
- [8] A. Tavakkol, P. Mehrvarzy, M. Arjomand, and H. Sarbazi-Azad, "Performance evaluation of dynamic page allocation strategies in ssds," *TOMPECS*, vol. 1, no. 2, pp. 1–33, 2016.
- [9] J. Zhang and M. Jung, "Zng: Architecting gpu multi-processors with new flash for scalable data analysis," in *ISCA*. IEEE, 2020, pp. 1064–1075.